

Contents

AI-Assisted Recruitment Decision Support Platform	1
The One-Line Pitch	1
1. The Problem	1
2. Why This Is the Headline Feature: Governing AI, Not Just Using It	1
3. Key Product Decisions (Where the PO Judgment Shows)	2
4. What Recruiters Actually See — Working Prototype	2
5. Selected User Stories	4
6. Competitive Landscape	4
7. Prioritization (MoSCoW)	5
8. Go-to-Market	6
9. Roadmap	6
10. Success Metrics	6
11. A Technical Note Worth Mentioning in Interviews	6

AI-Assisted Recruitment Decision Support Platform

A Product Case Study — Sameer

The One-Line Pitch

An AI system that ranks and explains candidate CVs for recruiters — but never auto-rejects, never penalizes CV formatting, and never lets application timing affect the outcome. Built as a working demonstration of how to govern AI output, not just call an API.

1. The Problem

- Recruiters get ~500 CVs per role and spend most of their time on repetitive first-pass screening, not high-judgment work
 - Manual screening is slow and inconsistent between reviewers
 - Legacy ATS keyword tools penalize CVs with non-standard formatting — good candidates get filtered out for reasons unrelated to skill
 - A subtler bias: recruiters unconsciously favor early applicants over later (even stronger) ones
 - **Regulatory context:** the EU AI Act classifies recruitment AI as high-risk, requiring human oversight and auditability — this isn't a hypothetical concern
-

2. Why This Is the Headline Feature: Governing AI, Not Just Using It

Most “AI screening” tools are a thin wrapper around an LLM call. This platform is built around a different idea: **the AI proposes, a human decides, and every step in between is traceable.**

Two things make this concrete, not just a claim:

1. Grounded explanations, not free-form narration. Every recommendation is built from specific extracted facts (“matched 4/6 required skills, missing X”) — not an open-ended LLM paragraph that sounds plausible but might not be accurate.

2. A real agentic workflow with a human checkpoint. When a recruiter asks “*Why does Candidate 7 need manual review?*”, the system runs a defined chain: retrieve extracted data → check extraction confidence → check missing requirements → retrieve the relevant policy → generate a grounded recommendation (Review / Proceed / Need More Information). If a step can’t retrieve what it needs, it says so — it doesn’t fabricate an answer.

The system never auto-rejects. Every candidate stays visible to a human. This is AI as a working system with governance built in — controlled inputs, traceable outputs, human review — not a chatbot bolted onto a workflow.

3. Key Product Decisions (Where the PO Judgment Shows)

Four stakeholders want different things, and the design has to hold all of them at once:

Stakeholder	Wants	Tension
HR Manager	Faster screening	vs. quality and fairness
Hiring Manager	A short, high-quality shortlist	vs. speed
Recruiter	Trust and explainability	vs. simplicity
Candidate	A fair shot regardless of formatting or timing	vs. automation efficiency

Two design decisions resolve this tension directly:

- **Dual-signal scoring** — extraction confidence is tracked *separately* from match score. A CV that couldn’t be reliably parsed is never silently ranked low — it’s routed to manual review, regardless of its score. This turns “AI might penalize non-standard CVs” from a risk into a designed, testable safeguard.
- **Batch-only ranking** — candidates are only ranked once the full batch is in, purely by score. Submission timestamp is logged for audit only, never used in scoring. A strong candidate applying near the deadline is judged exactly the same as one who applied on day one.

4. What Recruiters Actually See — Working Prototype

(This is no longer a wireframe — these are screenshots of the actual working Streamlit application, running on real (mocked) candidate data with a live LLM call behind “Why this rank?”)

Upload screen:

Candidates screen — ranked list with the agentic explanation expanded:

Notice the explanation isn’t a generic paragraph — it’s grounded, bulleted, and ends with an explicit recommendation, exactly matching the agentic chain described in Section 2.

A third screen (Hiring Tracker) was also built during implementation but isn’t pictured here — it’s a Kanban-style board (Shortlisted → Interview → Hired) showing each candidate’s contact info and a “View original CV” action, useful once a candidate moves past initial screening.

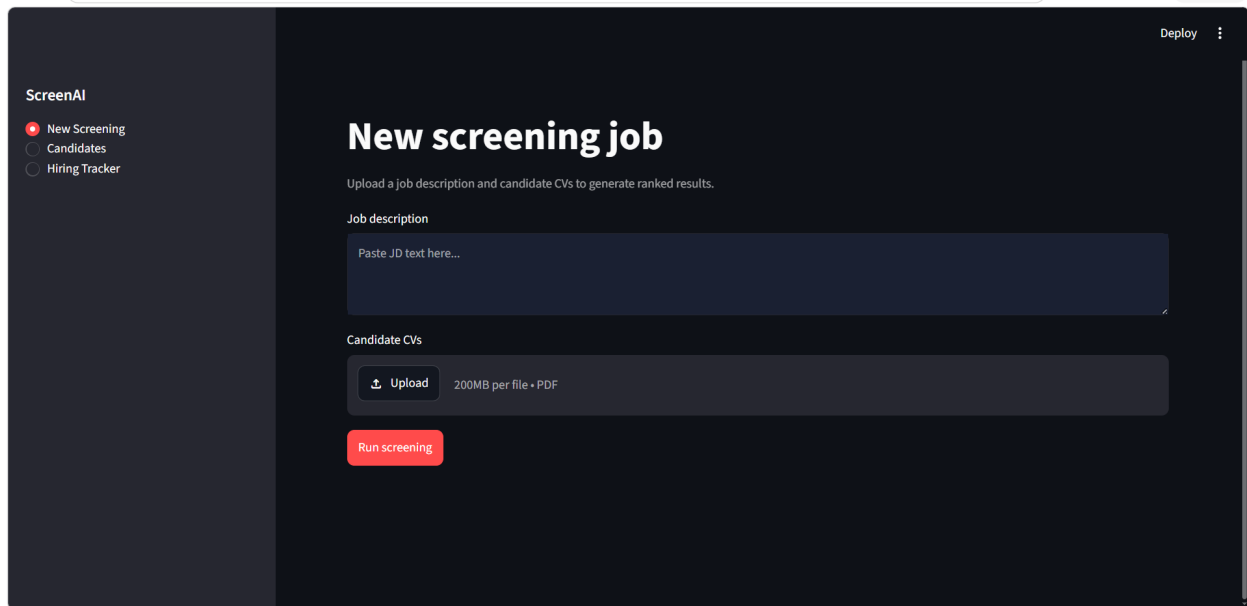


Figure 1: New screening upload screen

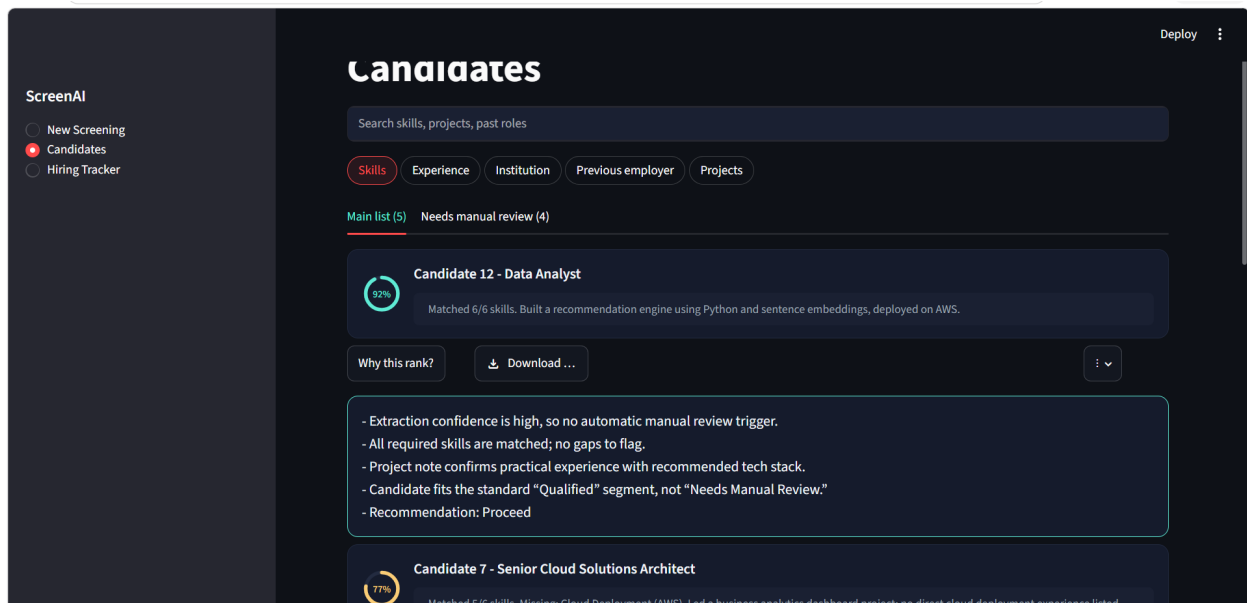


Figure 2: Candidates screen with explanation

What's real vs. what's UI-only in this build, stated plainly: - **Real:** search, tab filtering, candidate ranking by match score, the agentic explanation (a live call to an open-weight LLM via Groq), and the show/hide toggle on explanations - **UI-only (designed, not wired):** all 5 filter pills (Skills, Experience, Institution, Previous employer, Projects) are currently cosmetic — none of them change the results shown; ranking is always by match score regardless of pill selection. The “Download CV” / “View original CV” buttons, the “Schedule interview / Reject / Compare with others” actions, and the tracker’s “Move back a stage” action are all placeholders with no file or logic behind them. The “Run screening” button does not read the uploaded JD text or CV files — it always shows the same pre-loaded mock candidates regardless of what’s uploaded.

This is a deliberate scope decision for a 2-day prototype — the build prioritized proving the ranking + agentic-explanation logic over building every surrounding workflow action, and that tradeoff is disclosed here rather than left implicit.

5. Selected User Stories

- *As a recruiter*, I want ranked candidates with a clear explanation for each score, so I can trust and act on the recommendation quickly.
- *As a recruiter*, I want CVs that couldn't be reliably parsed flagged separately, so I don't overlook a strong candidate due to formatting.
- *As a recruiter*, I want to ask why a specific candidate needs manual review, so I get a traceable justification instead of digging through raw data myself.
- *As a candidate*, I want assurance that a formatting quirk in my CV won't cause me to be silently filtered out without human review.
- *As a compliance stakeholder*, I want every recommendation traceable to extracted facts and a documented rule, so decisions can be audited and defended.

(Full list of 20 user stories + acceptance criteria in the detailed BRD, linked at the end.)

6. Competitive Landscape

Approach	Speed	Consistency	Fairness	Auditability
Manual screening	Slow	Low — varies by reviewer	Medium — human bias still present	None — no structured record
Legacy ATS (Greenhouse, Lever, keyword-based tools)	Fast	High	Low — formatting-brittle, penalizes non-standard CVs	Low — rejects with no explanation
Ad hoc LLM use (recruiter pastes CV into ChatGPT)	Fast	Low — no structure between uses	Low — no safeguards	None — no audit trail
This platform	Fast	High	High — dual-signal design catches formatting bias	High — every score traceable to extracted facts and rule

The market gap this closes: established ATS platforms solved speed and consistency years ago, but they still treat formatting-brittleness and unexplained rejection as acceptable trade-offs. Ad hoc LLM use solves flexibility but adds no structure at all. Nobody in the current market combines all four properties — speed, consistency, fairness, and auditability — in one system. That combination, not any single feature, is the actual gap.

Why this matters commercially, not just technically: the EU AI Act’s high-risk classification for recruitment AI means the fairness/auditability gap isn’t a nice-to-have — vendors without a real answer to “how do you prevent formatting bias” or “can you show me why this candidate was ranked here” face growing regulatory exposure. This platform’s core design *is* the answer to those two questions.

7. Prioritization (MoSCoW)

Priority	Requirements	Why
Must	FR-1 to FR-11 (core extraction, dual-signal scoring, review-routing, grounded explanations, no-auto-reject, audit logging)	The core value proposition doesn’t exist without these — this is the platform
Must	FR-16 (agentic explanation workflow), FR-18 (synonym-aware matching), FR-20 (two-segment interface)	These directly demonstrate the platform’s core claims — AI governance, fairness, and visible safeguards. Without them, the differentiators are just described, not shown
Should	FR-12, FR-13 (skill filtering, adjustable weighting), FR-17 (deep project analysis), FR-23 (keyword search)	Meaningfully strengthen the product, and are realistic to build within the MVP window, but the platform still works without them
Could	FR-19 (extended filters: institution, employer, referral), FR-21 (per-candidate PDF download), FR-22 (recruiter notes), FR-25 (shortlist export)	Genuinely useful recruiter workflow features — included in requirements and the wireframe, but reasonable to defer past MVP without weakening the core pitch
Won’t (this round)	FR-24 (bulk shortlist), side-by-side candidate comparison	Real value, but lowest urgency relative to everything above — explicitly backlog, not overlooked

What actually gets built vs. only specified: Must-have items are built end-to-end in the working demo. Most Should and Could items are fully specified here and shown in the wireframe, but only a couple (search, and the two-segment toggle) are implemented in working code — a deliberate scope call given the build window, not a gap in the thinking.

8. Go-to-Market

Who this is for: - Mid-size companies (roughly 200-2,000 employees) with an internal recruitment team — large enough to feel the ~500-CVs-per-role volume problem, but without the budget or need for an enterprise ATS overhaul - Recruitment teams already frustrated with a legacy ATS's keyword-matching limitations, or currently doing ad hoc, unstructured LLM use with no safeguards

How adoption would start (not a company-wide rollout on day one): - Pilot with a single hiring team on a single high-volume role, run alongside their existing process (not replacing it yet) — the AI's recommendations are compared against what the team decided manually - Success in the pilot (faster shortlisting, no missed strong candidates in the review bucket) becomes the case for expanding to more roles/teams

How it competes for attention: - Positioned against legacy ATS tools as *"the fair, explainable alternative"* — directly targeting the two weaknesses named in the competitive analysis: formatting bias and no auditability - Positioned against ad hoc LLM use as *"the safe way to actually use AI for this"* — for teams already experimenting informally but worried about consistency or compliance - The EU AI Act angle becomes a genuine sales argument, not just a risk disclosure: compliance-conscious buyers (especially in regulated markets) have a concrete reason to prefer a system built with human-in-the-loop auditability from day one

9. Roadmap

Now (MVP): structured extraction → dual-signal scoring → agentic explanation workflow → basic Streamlit UI → local logging for audit

Next: real ATS integration, expanded bias auditing, candidate-facing transparency, role-based access

Later: multi-role comparison, full analytics dashboard, recruiter override tracking to improve matching over time

10. Success Metrics

- Screening time reduction (illustrative, clearly labeled as projected)
 - Review-bucket rate (are low-confidence CVs being caught — and are they being reviewed, not ignored)
 - Ranking consistency across repeated runs
 - Explanation groundedness (no fabricated details)
 - Bias swap-test outcome (identical CV, varied name/university → no score difference)
 - Order-independence check (identical CV, different timestamp → no score difference)
-

11. A Technical Note Worth Mentioning in Interviews

While building the agentic explanation feature, the LLM (an open-weight reasoning model served via Groq) occasionally returned a completely empty response with no error. Root cause: reasoning models spend part of their token budget on internal "thinking" before writing the actual answer — on harder candidates, that reasoning step sometimes consumed the entire token budget, leaving nothing for the actual explanation.

Fix: lowered the reasoning effort setting (since this task doesn't need deep reasoning) and raised the token budget, plus added a safety check so a genuinely empty response shows a clear retry message instead of silently failing.

This is a small thing, but it's a real example of diagnosing a subtle production-AI failure mode rather than just prompting a model and hoping — worth having ready if asked “what was technically hardest.”

Full detailed requirements, all 25+ functional requirements, complete user story set, and acceptance criteria: see the accompanying Business Requirements & Solution Design Document (v2).